Cognizant - UpGrad Training Program
WEEKLY CODING SUBMISSION

Name	**:**	**Durgesh Vijay Bhangale**
Week	**:**	**Week 6**
Batch	**:**	**B1_Python**
Trainer	**:**	**Kunal**

Day 1, 2(23-3-26 – 24-3-26)

SmartSkill Exchange Platform using ASP.NET Core 8 & EF Core 8

Models/User.cs

```
public class User {
   public int Id { get; set; }
   [Required] public string Name { get; set; }
   [Required, EmailAddress] public string Email { get; set; }
   public List<Skill> Skills { get; set; }
   public Wallet Wallet { get; set; }
}
```

Skill.cs

```
public class Skill {
   public int Id { get; set; }
   [Required, StringLength(50)] public string Title { get; set; }
   public string Description { get; set; }
   public int UserId { get; set; }
   public User User { get; set; }
   public List<Booking> Bookings { get; set; }
}
```

Review.cs

```
public class Review {
   public int Id { get; set; }
   public string Comment { get; set; }
   public int Rating { get; set; }
   public int SkillId { get; set; }
   public Skill Skill { get; set; }
}
```

Booking.cs

```
public class Booking {
   public int Id { get; set; }
   public int SkillId { get; set; }
   public Skill Skill { get; set; }
   public int UserId { get; set; }
   public User User { get; set; }
}
```

Wallet.cs

```
public class Wallet {
   public int Id { get; set; }
   public int Points { get; set; }
   public int UserId { get; set; }
```

```
    public User User { get; set; }
}
```

Controller

AccountController.cs

```
using Microsoft.AspNetCore.Mvc;

public class AccountController : Controller
{
    private readonly ApplicationDbContext _context;
    public AccountController(ApplicationDbContext context) { _context = context; }

    [HttpGet]
    public IActionResult Register() => View();


    [HttpPost]
    public IActionResult Register(User user)
    {
        if (ModelState.IsValid)
        {
            _context.Users.Add(user);
            _context.SaveChanges();

            _context.Wallets.Add(new Wallet { UserId = user.Id, Points = 50 });
            _context.SaveChanges();

            TempData["Success"] = "Registration successful!";
            return RedirectToAction("Login");
        }
        return View(user);
    }

    [HttpGet]
    public IActionResult Login() => View();

    [HttpPost]
    public IActionResult Login(string email)
    {
        var user = _context.Users.FirstOrDefault(u => u.Email == email);
        if (user != null)
        {
            HttpContext.Session.SetInt32("UserId", user.Id);
            TempData["Success"] = "Login successful!";
            return RedirectToAction("Index", "Skill");
        }
```

```
        TempData["Error"] = "Invalid login!";
        return View();
    }
}
```

AdminController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

public class AdminController : Controller
{
    private readonly ApplicationDbContext _context;
    public AdminController(ApplicationDbContext context) { _context = context; }

    public IActionResult Users() => View(_context.Users.ToList());

    public IActionResult Bookings() => View(
        _context.Bookings.Include(b => b.Skill).Include(b => b.User).ToList()
    );
}
```

BookingController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

public class BookingController : Controller
{
    private readonly ApplicationDbContext _context;
    public BookingController(ApplicationDbContext context) { _context = context; }

    public IActionResult Index()
    {
        var bookings = _context.Bookings
            .Include(b => b.Skill)
            .Include(b => b.User)
            .ToList();
        return View(bookings);
    }


    [HttpGet]
    public IActionResult Create() => View();

    [HttpPost]
    public IActionResult Create(Booking booking)
    {
        if (ModelState.IsValid)
```

```
        {
            var wallet = _context.Wallets.FirstOrDefault(w => w.UserId == booking.UserId);
            if (wallet != null && wallet.Points >= 10)
            {
                wallet.Points -= 10;
                _context.Bookings.Add(booking);
                _context.SaveChanges();
                TempData["Success"] = "Booking created!";
                return RedirectToAction("Index");
            }
            TempData["Error"] = "Not enough points!";
        }
        return View(booking);
    }
}
```

SkillController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

public class SkillController : Controller
{
    private readonly ApplicationDbContext _context;
    public SkillController(ApplicationDbContext context) { _context = context; }

    public IActionResult Index()
    {
        var skills = _context.Skills.Include(s => s.User).ToList();
        return View(skills);
    }

    [HttpGet]
    public IActionResult Create() => View();

    [HttpPost]
    public IActionResult Create(Skill skill)
    {
        if (ModelState.IsValid)
        {
            _context.Skills.Add(skill);
            _context.SaveChanges();
            TempData["Success"] = "Skill added!";
            return RedirectToAction("Index");
        }
        return View(skill);
    }
```

```
    [HttpGet]
    public IActionResult Edit(int id)
    {
        var skill = _context.Skills.Find(id);
        if (skill == null) return NotFound();
        return View(skill);
    }

    [HttpPost]
    public IActionResult Edit(Skill skill)
    {
        if (ModelState.IsValid)
        {
            _context.Skills.Update(skill);
            _context.SaveChanges();
            TempData["Success"] = "Skill updated!";
            return RedirectToAction("Index");
        }
        return View(skill);
    }

    [HttpGet]
    public IActionResult Delete(int id)
    {
        var skill = _context.Skills.Find(id);
        if (skill == null) return NotFound();
        return View(skill);
    }

    [HttpPost, ActionName("Delete")]
    public IActionResult DeleteConfirmed(int id)
    {
        var skill = _context.Skills.Find(id);
        if (skill != null)
        {
            _context.Skills.Remove(skill);
            _context.SaveChanges();
            TempData["Success"] = "Skill deleted!";
        }
        return RedirectToAction("Index");
    }
}
```

Views/Skill

Index.cshtml

```
@model List<Skill>
<h2>Skills</h2>
```

```
@if(TempData["Success"] != null) {
   <p style="color:green">@TempData["Success"]</p>
}
<table class="table">
   <tr>
      <th>Title</th>
      <th>Description</th>
      <th>User</th>
      <th>Actions</th>
   </tr>
   @foreach(var s in Model) {
      <tr>
         <td>@s.Title</td>
         <td>@s.Description</td>
         <td>@s.User?.Name</td>
         <td>
            <a asp-action="Edit" asp-route-id="@s.Id">Edit</a> |
            <a asp-action="Delete" asp-route-id="@s.Id">Delete</a>
         </td>
      </tr>
   }
</table>

<a asp-action="Create" class="btn btn-primary">Add Skill</a>
```

Create.cshtml

```
@model Skill
<h2>Add Skill</h2>

<form asp-action="Create" method="post">
   <div>
      <label>Title:</label>
      <input asp-for="Title" />
      <span asp-validation-for="Title"></span>
   </div>
   <div>
      <label>Description:</label>
      <input asp-for="Description" />
   </div>
   <button type="submit" class="btn btn-success">Save</button>
</form>
```

Edit.cshtml

```
@model Skill
<h2>Edit Skill</h2>

<form asp-action="Edit" method="post">
```

```
    <input type="hidden" asp-for="Id" />
    <div>
        <label>Title:</label>
        <input asp-for="Title" />
    </div>
    <div>
        <label>Description:</label>
        <input asp-for="Description" />
    </div>
    <button type="submit" class="btn btn-warning">Update</button>
</form>
```

Delete.cshtml

```
@model Skill
<h2>Delete Skill</h2>
<p>Are you sure you want to delete <b>@Model.Title</b>?</p>

<form asp-action="Delete" method="post">
    <input type="hidden" asp-for="Id" />
    <button type="submit" class="btn btn-danger">Delete</button>
    <a asp-action="Index" class="btn btn-secondary">Cancel</a>
</form>
```

📁 Views/Booking

Index.cshtml

```
@model List<Booking>
<h2>Bookings</h2>

@if(TempData["Success"] != null) { <p style="color:green">@TempData["Success"]</p> }
@if(TempData["Error"] != null) { <p style="color:red">@TempData["Error"]</p> }

<table class="table">
    <tr>
        <th>Skill</th>
        <th>User</th>
    </tr>
    @foreach(var b in Model) {
        <tr>
            <td>@b.Skill?.Title</td>
            <td>@b.User?.Name</td>
        </tr>
    }
</table>
<a asp-action="Create" class="btn btn-primary">Book Session</a>
@model Booking
<h2>Create Booking</h2>
```

```
<form asp-action="Create" method="post">
   <div>
      <label>Skill Id:</label>
      <input asp-for="SkillId" />
   </div>
   <div>
      <label>User Id:</label>
      <input asp-for="UserId" />
   </div>
   <button type="submit" class="btn btn-success">Book</button>
</form>
```

Views/Account

Register.cshtml

```
@model User
<h2>Register</h2>

<form asp-action="Register" method="post">
   <div>
      <label>Name:</label>
      <input asp-for="Name" />
   </div>
   <div>
      <label>Email:</label>
      <input asp-for="Email" />
   </div>
   <button type="submit" class="btn btn-success">Register</button>
</form>
```

Login.cshtml

```
<h2>Login</h2>

<form asp-action="Login" method="post">
   <div>
      <label>Email:</label>
      <input type="text" name="email" />
   </div>
   <button type="submit" class="btn btn-primary">Login</button>
</form>
```

Views/Shared

_Layout.cshtml

```
<!DOCTYPE html>
<html>
<head>
   <title>SmartSkill Exchange</title>
   <link rel="stylesheet" href="~/css/site.css" />
```

```
</head>
<body>
   <nav>
      <a asp-controller="Skill" asp-action="Index">Skills</a> |
      <a asp-controller="Booking" asp-action="Index">Bookings</a> |
      <a asp-controller="Account" asp-action="Register">Register</a> |
      <a asp-controller="Account" asp-action="Login">Login</a>
   </nav>
   <div class="container">
      @RenderBody()
   </div>
</body>
</html>
```

Data

ApplicationDbContext.cs

```
public class ApplicationDbContext : DbContext {
   public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) : base(options) { }
   public DbSet<User> Users { get; set; }
   public DbSet<Skill> Skills { get; set; }
   public DbSet<Booking> Bookings { get; set; }
   public DbSet<Review> Reviews { get; set; }
   public DbSet<Wallet> Wallets { get; set; }
}
```

Program.cs

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();
builder.Services.AddDbContext<ApplicationDbContext>(options =>
   options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
builder.Services.AddSession();

var app = builder.Build();
app.UseSession();
app.UseStaticFiles();
app.MapDefaultControllerRoute();
app.Run();
```

Day 3(25-3-26)

Assignment: Order Management API

Model/order.cs

```
public enum OrderStatus { Pending, HighValueOrder, Completed, Cancelled }

public class Order {
   public int Id { get; set; }
   public string CustomerName { get; set; }
   public double TotalAmount { get; set; }
   public OrderStatus Status { get; set; }
}
```

DTO/OrderDtos.cs

```
public class OrderDto {
   public string CustomerName { get; set; }
   public double TotalAmount { get; set; }
}
```

Data/AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;

public class AppDbContext : DbContext {
   public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
   public DbSet<Order> Orders { get; set; }
}
```

Repositories/IOrederRepository.cs

```
public interface IOrderRepository {
   Task<IEnumerable<Order>> GetAllOrders();
   Task<Order?> GetOrderById(int id);
   Task<Order> CreateOrder(Order order);
   Task<bool> UpdateStatus(int id, OrderStatus status);
   Task<bool> DeleteOrder(int id);
}
```

OrderRepositories.cs

```
using Microsoft.EntityFrameworkCore;

public class OrderRepository : IOrderRepository {
   private readonly AppDbContext _context;
   public OrderRepository(AppDbContext context) { _context = context; }

   public async Task<IEnumerable<Order>> GetAllOrders() => await _context.Orders.ToListAsync();

   public async Task<Order?> GetOrderById(int id) => await _context.Orders.FindAsync(id);
```

```
    public async Task<Order> CreateOrder(Order order) {
        order.Status = order.TotalAmount > 50000 ? OrderStatus.HighValueOrder : OrderStatus.Pending;
        _context.Orders.Add(order);
        await _context.SaveChangesAsync();
        return order;
    }

    public async Task<bool> UpdateStatus(int id, OrderStatus status) {
        var order = await _context.Orders.FindAsync(id);
        if (order == null) return false;
        order.Status = status;
        await _context.SaveChangesAsync();
        return true;
    }

    public async Task<bool> DeleteOrder(int id) {
        var order = await _context.Orders.FindAsync(id);
        if (order == null) return false;
        _context.Orders.Remove(order);
        await _context.SaveChangesAsync();
        return true;
    }
}
```

MappinProfiles.cs

```
using AutoMapper;

public class MappingProfile : Profile {
    public MappingProfile() {
        CreateMap<OrderDto, Order>();
    }
}
```

Controller/OrderController.cs

```
using AutoMapper;
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class OrderController : ControllerBase {
    private readonly IOrderRepository _repo;
    private readonly IMapper _mapper;

    public OrderController(IOrderRepository repo, IMapper mapper) {
        _repo = repo; _mapper = mapper;
    }

    [HttpGet]
    public async Task<IActionResult> GetAll() => Ok(await _repo.GetAllOrders());
```

```
    [HttpGet("{id}")]
    public async Task<IActionResult> Get(int id) {
        var order = await _repo.GetOrderById(id);
        return order == null ? NotFound() : Ok(order);
    }

    [HttpPost]
    public async Task<IActionResult> Create(OrderDto dto) {
        if (string.IsNullOrEmpty(dto.CustomerName) || dto.TotalAmount <= 0)
            return BadRequest("Invalid input");

        var order = _mapper.Map<Order>(dto);
        var created = await _repo.CreateOrder(order);
        return CreatedAtAction(nameof(Get), new { id = created.Id }, created);
    }

    [HttpPut("{id}")]
    public async Task<IActionResult> UpdateStatus(int id, [FromBody] string status) {
        if (!Enum.TryParse<OrderStatus>(status, true, out var parsed))
            return BadRequest("Invalid status");

        var success = await _repo.UpdateStatus(id, parsed);
        return success ? NoContent() : NotFound();
    }

    [HttpDelete("{id}")]
    public async Task<IActionResult> Delete(int id) {
        var success = await _repo.DeleteOrder(id);
        return success ? NoContent() : NotFound();
    }
}
```

Middleware/ExceptionMiddlewares.cs

```
public class ExceptionMiddleware {
    private readonly RequestDelegate _next;
    public ExceptionMiddleware(RequestDelegate next) { _next = next; }

    public async Task InvokeAsync(HttpContext context) {
        try {
            await _next(context);
        } catch (Exception ex) {
            context.Response.StatusCode = 500;
            context.Response.ContentType = "application/json";
            await context.Response.WriteAsync($"{{\"error\":\"Something went wrong\"}}");
        }
    }
}
```

Services/OrderBackgroundService.cs

```
public class OrderBackgroundService : BackgroundService {
```

```
    protected override async Task ExecuteAsync(CancellationToken stoppingToken) {
        while (!stoppingToken.IsCancellationRequested) {
            Console.WriteLine("Checking pending orders...");
            await Task.Delay(10000, stoppingToken);
        }
    }
}
```

Program.cs

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.AspNetCore.Builder;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseInMemoryDatabase("OrdersDb"));
builder.Services.AddDbContext<AppDbContext>(opt => opt.UseInMemoryDatabase("OrdersDb"));
builder.Services.AddScoped<IOrderRepository, OrderRepository>();
builder.Services.AddAutoMapper(typeof(Program));
builder.Services.AddHostedService<OrderBackgroundService>();
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

app.UseMiddleware<ExceptionMiddleware>();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.MapControllers();
app.Run();
```

Day 4(26-3-26)

Question 1: Area & Perimeter of Rectangle

```
using System;

public class Shape
{
   public int Length { get; set; }
   public int Width { get; set; }

   public Shape(int length, int width)
   {
      Length = length;
      Width = width;
   }
}

public class Rectangle : Shape
{
   public Rectangle(int length, int width) : base(length, width) { }

   public int CalculateArea()
   {
      return Length * Width;
   }

   public int CalculatePerimeter()
   {
      return 2 * (Length + Width);
   }
}

class Program
{
   static void Main(string[] args)
   {

      var input = Console.ReadLine().Split(' ');
      int length = int.Parse(input[0]);
      int width = int.Parse(input[1]);

      Rectangle rect = new Rectangle(length, width);

      Console.WriteLine("Area: " + rect.CalculateArea());
      Console.WriteLine("Perimeter: " + rect.CalculatePerimeter());
   }
}
```

Question 2: Supermarket Store (OOP)

```
using System;
using System.Collections.Generic;

public class Product {
    public string Name { get; set; }
    public double Price { get; set; }
    public int Quantity { get; set; }

    public Product(string name, double price, int quantity) {
        Name = name; Price = price; Quantity = quantity;
    }

    public virtual string GetProductDetails() {
        return $"{Name} - Price: {Price}, Quantity: {Quantity}";
    }
}

public class Electronics : Product {
    public int WarrantyPeriod { get; set; }

    public Electronics(string name, double price, int quantity, int warranty)
        : base(name, price, quantity) {
        WarrantyPeriod = warranty;
    }

    public override string GetProductDetails() {
        return $"{Name} - Price: {Price}, Quantity: {Quantity}, Warranty: {WarrantyPeriod} months";
    }
}

public class Clothing : Product {
    public string Size { get; set; }

    public Clothing(string name, double price, int quantity, string size)
        : base(name, price, quantity) {
        Size = size;
    }

    public override string GetProductDetails() {
        return $"{Name} - Price: {Price}, Quantity: {Quantity}, Size: {Size}";
    }
}

public class Inventory {
    private List<Product> products = new List<Product>();

    public void AddProduct(Product product) {
        products.Add(product);
        Console.WriteLine($"Product added to inventory: {product.Name}");
    }
```

```
    public void DisplayInventory() {
        Console.WriteLine("\nInventory:");
        foreach (var p in products) {
            Console.WriteLine(p.GetProductDetails());
        }
    }

    public void CalculateTotalValue() {
        double total = 0;
        foreach (var p in products) {
            total += p.Price * p.Quantity;
        }
        Console.WriteLine($"\nTotal value of the inventory: {total:F2}");
    }
}

class Program {
    static void Main() {
        int num = int.Parse(Console.ReadLine());
        Inventory inventory = new Inventory();

        for (int i = 0; i < num; i++) {
            var input = Console.ReadLine().Split(", ");
            string type = input[0];
            string name = input[1];
            double price = double.Parse(input[2]);
            int quantity = int.Parse(input[3]);

            if (type == "Electronics") {
                int warranty = int.Parse(input[4]);
                inventory.AddProduct(new Electronics(name, price, quantity, warranty));
            } else if (type == "Clothing") {
                string size = input[4];
                inventory.AddProduct(new Clothing(name, price, quantity, size));
            }
        }

        inventory.DisplayInventory();
        inventory.CalculateTotalValue();
    }
}
```

Question 3: Virus Resolving

```
using System;

class Program {
    static int VirusResolving(int n, int x, int y, int[] damages) {
        int pairs = n / 2;
        int leftover = n % 2;

        int timeWithPairs = pairs * Math.Min(x, 2 * y) + leftover * y;

        return timeWithPairs;
    }

    static void Main() {
        var input = Console.ReadLine().Split(' ');
        int n = int.Parse(input[0]);
        int x = int.Parse(input[1]);
        int y = int.Parse(input[2]);

        var damages = Array.ConvertAll(Console.ReadLine().Split(' '), int.Parse);

        Console.WriteLine(VirusResolving(n, x, y, damages));
    }
}
```

Day 5(27-3-26)

Problem-1

Models/StudentScore.cs

```
namespace ExamApp.Models
{
    public class StudentScore
    {
        public string StudentName { get; set; }
        public int ObtainedMarks { get; set; }
    }
}
```

Controllers/StudentController.cs

```
using Microsoft.AspNetCore.Mvc;
using ExamApp.Models;

namespace ExamApp.Controllers
{
    public class StudentController : Controller
    {
        public IActionResult Entry()
        {
            return View();
        }

        [HttpPost]
        public IActionResult Display(StudentScore score)
        {
            return View(score);
        }
    }
}
```

Views/Student/Entry.cshtml

```
@model ExamApp.Models.StudentScore
<h2>Fill Student Information</h2>
<form asp-action="Display" method="post">
    <input asp-for="StudentName" placeholder="Enter Student Name" />
    <br /><br />
    <input asp-for="ObtainedMarks" type="number" placeholder="Enter Marks" />
    <br /><br />
    <button type="submit">Submit</button>
</form>
```

Problem 2

Middleware/ExecutionTimeMiddleware.cs

```
using System.Diagnostics;

public class ExecutionTimeMiddleware
{
    private readonly RequestDelegate _next;

    public ExecutionTimeMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task Invoke(HttpContext context)
    {
        var timer = Stopwatch.StartNew();
        await _next(context);
        timer.Stop();
        context.Response.Headers["X-Execution-Time"] = $"{timer.ElapsedMilliseconds}ms";
    }
}
```

Program.cs

```
using ExamApp.Middleware;

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.UseMiddleware<ExecutionTimeMiddleware>();
app.MapGet("/", () => "Custom Middleware Active!");

app.Run();
```

Problem 3

Models/Staff.cs

```
namespace ExamApp.Models
{
    public class Staff
    {
        public int StaffId { get; set; }
```

```
        public string FullName { get; set; }
        public double Pay { get; set; }
        public string Division { get; set; }
    }
}
```

Data/DatabaseContext.cs

```
using Microsoft.EntityFrameworkCore;
using ExamApp.Models;

namespace ExamApp.Data
{
    public class DatabaseContext : DbContext
    {
        public DatabaseContext(DbContextOptions<DatabaseContext> options)
            : base(options) { }

        public DbSet<Staff> StaffMembers { get; set; }
    }
}
```

Controllers/staffController.cs

```
using Microsoft.AspNetCore.Mvc;
using ExamApp.Data;
using System.Linq;

namespace ExamApp.Controllers
{
    public class StaffController : Controller
    {
        private readonly DatabaseContext _db;

        public StaffController(DatabaseContext db)
        {
            _db = db;
        }

        public IActionResult Index()
        {
            var employees = _db.StaffMembers
                .Select(s => new { s.FullName, s.Division })
                .ToList();

            return Json(employees);
        }
```

```
    }
}
```

Program.cs

```
using Microsoft.EntityFrameworkCore;
using ExamApp.Data;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();
builder.Services.AddDbContext<DatabaseContext>(options =>
    options.UseInMemoryDatabase("StaffDB"));

var app = builder.Build();
app.UseRouting();
app.MapControllers();
app.Run();
```

Problem 4

Models/Item.cs

```
namespace ExamApp.Models
{
    public class Item
    {
        public string Title { get; set; }
        public int Cost { get; set; }
    }
}
```

Controllers/ItemController.cs

```
using Microsoft.AspNetCore.Mvc;
using ExamApp.Models;
using System.Linq;

namespace ExamApp.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ItemsController : ControllerBase
    {
        private static List<Item> inventory = new List<Item>
        {
            new Item { Title = "Laptop", Cost = 500 },
            new Item { Title = "Phone", Cost = 300 },
            new Item { Title = "Tablet", Cost = 150 }
```

```
        };

        [HttpGet]
        public IActionResult Get(int? minCost, int? maxCost)
        {
            var query = inventory.AsQueryable();
            if (minCost.HasValue)
                query = query.Where(i => i.Cost >= minCost.Value);
            if (maxCost.HasValue)
                query = query.Where(i => i.Cost <= maxCost.Value);

            return Ok(query.ToList());
        }
    }
}
```

Program.cs

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();

var app = builder.Build();
app.MapControllers();
app.Run();
```

Problem 5

Controllers/MainController.cs

```
using Microsoft.AspNetCore.Mvc;

namespace ExamApp.Controllers
{
    public class MainController : Controller
    {
        public IActionResult Index()
        {
            int visits = 1;
            if (Request.Cookies["pageVisits"] != null)
            {
                visits = int.Parse(Request.Cookies["pageVisits"]) + 1;
            }

            Response.Cookies.Append("pageVisits", visits.ToString());
            ViewBag.Visits = visits;
```

```
            return View();
        }
    }
}
```

Views/Main/Index.cshtml

```
@{
    ViewData["Title"] = "Visit Tracker";
}
<!DOCTYPE html>
<html>
<head>
    <title>Visit Counter</title>
</head>
<body>
    <h2>You have opened this page @ViewBag.Visits times</h2>
</body>
</html>
```

Program.cs

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();

var app = builder.Build();
app.UseRouting();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Main}/{action=Index}/{id?}");

app.Run();
```